

Show that after all edges are processed by CONNECTED-COMPONENTS, two vertices belong to the same connected component if and only if they belong to the same set.

19.1-3

During the execution of CONNECTED-COMPONENTS on an undirected graph $G = (V, E)$ with k connected components, how many times is FIND-SET called? How many times is UNION called? Express your answers in terms of $|V|$, $|E|$, and k .

19.2 Linked-list representation of disjoint sets

Figure 19.2(a) shows a simple way to implement a disjoint-set data structure: each set is represented by its own linked list. The object for each set has attributes *head*, pointing to the first object in the list, and *tail*, pointing to the last object. Each object in the list contains a set member, a pointer to the next object in the list, and a pointer back to the set object. Within each linked list, the objects may appear in any order. The representative is the set member in the first object in the list.

With this linked-list representation, both MAKE-SET and FIND-SET require only $O(1)$ time. To carry out MAKE-SET(x), create a new linked list whose only object is x . For FIND-SET(x), just follow the pointer from x back to its set object and then return the member in the object that *head* points to. For example, in Figure 19.2(a), the call FIND-SET(g) returns f .

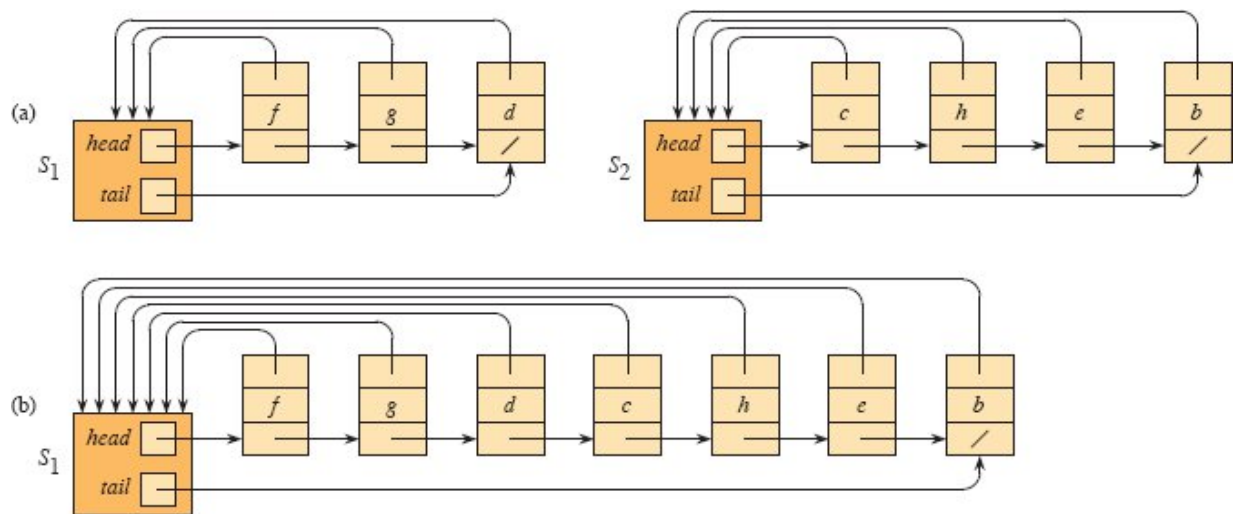


Figure 19.2 (a) Linked-list representations of two sets. Set S_1 contains members d , f , and g , with representative f , and set S_2 contains members b , c , e , and h , with representative c . Each object in the list contains a set member, a pointer to the next object in the list, and a pointer back to the set object. Each set object has pointers *head* and *tail* to the first and last objects, respectively. **(b)** The result of $\text{UNION}(g, e)$, which appends the linked list containing e to the linked list containing g . The representative of the resulting set is f . The set object for e 's list, S_2 , is destroyed.

A simple implementation of union

The simplest implementation of the UNION operation using the linked-list set representation takes significantly more time than MAKE-SET or FIND-SET. As Figure 19.2(b) shows, the operation $\text{UNION}(x, y)$ appends y 's list onto the end of x 's list. The representative of x 's list becomes the representative of the resulting set. To quickly find where to append y 's list, use the *tail* pointer for x 's list. Because all members of y 's list join x 's list, the UNION operation destroys the set object for y 's list. The UNION operation is where this implementation pays the price for FIND-SET taking constant time: UNION must also update the pointer to the set object for each object originally on y 's list, which takes time linear in the length of y 's list. In Figure 19.2, for example, the operation $\text{UNION}(g, e)$ causes pointers to be updated in the objects for b , c , e , and h .

In fact, we can construct a sequence of m operations on n objects that requires $\Theta(n^2)$ time. Starting with objects $x_1, x_2, \dots, x_n$, execute the sequence of n MAKE-SET operations followed by $n - 1$ UNION operations shown in Figure 19.3, so that $m = 2n - 1$. The n MAKE-SET operations take $\Theta(n)$ time. Because the i th UNION operation updates i objects, the total number of objects updated by all $n - 1$ UNION operations forms an arithmetic series:

Operation	Number of objects updated
MAKE-SET(x_1)	1
MAKE-SET(x_2)	1
$\vdots$	$\vdots$
MAKE-SET(x_n)	1
UNION(x_2, x_1)	1
UNION(x_3, x_2)	2
UNION(x_4, x_3)	3
$\vdots$	$\vdots$
UNION(x_n, x_{n-1})	$n - 1$

Figure 19.3 A sequence of $2n - 1$ operations on n objects that takes $\Theta(n^2)$ time, or $\Theta(n)$ time per operation on average, using the linked-list set representation and the simple implementation of UNION.

$$\sum_{i=1}^{n-1} i = \Theta(n^2) .$$

The total number of operations is $2n - 1$, and so each operation on average requires $\Theta(n)$ time. That is, the amortized time of an operation is $\Theta(n)$.

A weighted-union heuristic

In the worst case, the above implementation of UNION requires an average of $\Theta(n)$ time per call, because it might be appending a longer list onto a shorter list, and the procedure must update the pointer to the set object for each member of the longer list. Suppose instead that each list also includes the length of the list (which can be maintained straightforwardly with constant overhead) and that the UNION

procedure always appends the shorter list onto the longer, breaking ties arbitrarily. With this simple *weighted-union heuristic*, a single UNION operation can still take $\Omega(n)$ time if both sets have $\Omega(n)$ members. As the following theorem shows, however, a sequence of m MAKE-SET, UNION, and FIND-SET operations, n of which are MAKE-SET operations, takes $O(m + n \lg n)$ time.

Theorem 19.1

Using the linked-list representation of disjoint sets and the weighted-union heuristic, a sequence of m MAKE-SET, UNION, and FIND-SET operations, n of which are MAKE-SET operations, takes $O(m + n \lg n)$ time.

Proof Because each UNION operation unites two disjoint sets, at most $n - 1$ UNION operations occur over all. We now bound the total time taken by these UNION operations. We start by determining, for each object, an upper bound on the number of times the object's pointer back to its set object is updated. Consider a particular object x . Each time x 's pointer is updated, x must have started in the smaller set. The first time x 's pointer is updated, therefore, the resulting set must have at least 2 members. Similarly, the next time x 's pointer is updated, the resulting set must have had at least 4 members. Continuing on, for any $k \leq n$, after x 's pointer has been updated $\lceil \lg k \rceil$ times, the resulting set must have at least k members. Since the largest set has at most n members, each object's pointer is updated at most $\lceil \lg n \rceil$ times over all the UNION operations. Thus the total time spent updating object pointers over all UNION operations is $O(n \lg n)$. We must also account for updating the *tail* pointers and the list lengths, which take only $\Theta(1)$ time per UNION operation. The total time spent in all UNION operations is thus $O(n \lg n)$.

The time for the entire sequence of m operations follows. Each MAKE-SET and FIND-SET operation takes $O(1)$ time, and there are $O(m)$ of them. The total time for the entire sequence is thus $O(m + n \lg n)$.

■

Exercises

19.2-1

Write pseudocode for MAKE-SET, FIND-SET, and UNION using the linked-list representation and the weighted-union heuristic. Make sure to specify the attributes that you assume for set objects and list objects.

19.2-2

Show the data structure that results and the answers returned by the FIND-SET operations in the following program. Use the linked-list representation with the weighted-union heuristic. Assume that if the sets containing x_i and x_j have the same size, then the operation $\text{UNION}(x_i, x_j)$ appends x_j 's list onto x_i 's list.

```
1 for  $i = 1$  to 16
2   MAKE-SET( $x_i$ )
3 for  $i = 1$  to 15 by 2
4   UNION( $x_i, x_{i+1}$ )
5 for  $i = 1$  to 13 by 4
6   UNION( $x_i, x_{i+2}$ )
7 UNION( $x_1, x_5$ )
8 UNION( $x_{11}, x_{13}$ )
9 UNION( $x_1, x_{10}$ )
10 FIND-SET( $x_2$ )
11 FIND-SET( $x_9$ )
```

19.2-3

Adapt the aggregate proof of Theorem 19.1 to obtain amortized time bounds of $O(1)$ for MAKE-SET and FIND-SET and $O(\lg n)$ for UNION using the linked-list representation and the weighted-union heuristic.

19.2-4

Give a tight asymptotic bound on the running time of the sequence of operations in Figure 19.3 assuming the linked-list representation and the

weighted-union heuristic.

19.2-5

Professor Gompers suspects that it might be possible to keep just one pointer in each set object, rather than two (*head* and *tail*), while keeping the number of pointers in each list element at two. Show that the professor's suspicion is well founded by describing how to represent each set by a linked list such that each operation has the same running time as the operations described in this section. Describe also how the operations work. Your scheme should allow for the weighted-union heuristic, with the same effect as described in this section. (*Hint*: Use the tail of a linked list as its set's representative.)

19.2-6

Suggest a simple change to the UNION procedure for the linked-list representation that removes the need to keep the *tail* pointer to the last object in each list. Regardless of whether the weighted-union heuristic is used, your change should not change the asymptotic running time of the UNION procedure. (*Hint*: Rather than appending one list to another, splice them together.)

19.3 Disjoint-set forests

A faster implementation of disjoint sets represents sets by rooted trees, with each node containing one member and each tree representing one set. In a *disjoint-set forest*, illustrated in Figure 19.4(a), each member points only to its parent. The root of each tree contains the representative and is its own parent. As we'll see, although the straightforward algorithms that use this representation are no faster than ones that use the linked-list representation, two heuristics—"union by rank" and "path compression"—yield an asymptotically optimal disjoint-set data structure.

The three disjoint-set operations have simple implementations. A MAKE-SET operation simply creates a tree with just one node. A FIND-SET operation follows parent pointers until it reaches the root of the tree. The nodes visited on this simple path toward the root constitute